



دانشکده مهندسی کامپیوتر

عنوان: گزارش پروژه درخت تصمیم

توضیحات: تشخیص افسردگی دانشجویان با درخت تصمیم

نام استاد: دکتر عبدی

نام دانشجو: امیرحسین اسلامی

بهار ۱۴۰۴

چکیده

در این پروژه، هدف آموزش یک مدل درخت تصمیم برای تشخیص افسردگی دانشجویان با استفاده از زبان پایتون و کتابخانه NumPy است. در ابتدا، یک دیتاست مناسب از وبسایت Kaggle انتخاب شد که شامل تعداد زیادی نمونه و ویژگی‌های مرتبط با بیماری افسردگی دانشجویان بود. سپس فرآیند پیش‌پردازش داده‌ها انجام شد که شامل حذف مقادیر پرت، تبدیل داده‌های متنی به عددی، گروه‌بندی ویژگی‌های پیوسته و کاهش ابعاد مجموعه داده بود.

پس از آماده‌سازی داده‌ها، مجموعه داده به دو بخش آموزشی و تست تقسیم شد. سپس مدل درخت تصمیم با تنظیم مقادیر عمق درخت، حداقل نمونه جداسازی و نسبت داده‌های آموزشی به تست بهینه‌سازی شد تا از بیش‌برازش جلوگیری شود و دقت مدل در داده‌های تست افزایش یابد. در نهایت، مدل نهایی با بهترین تنظیمات آموزش داده شد و عملکرد آن بر اساس معیار دقت و کاهش بیش‌برازش بررسی گردید.

واژه‌های کلیدی: درخت تصمیم، پردازش داده، یادگیری ماشین، تحلیل داده

فهرست مطالب

۷	فصل ۱: انتخاب و اصلاح اولیه دیتاست
Error! Bookmark not defined.	1-1- انتخاب دیتاست
۸	۱-۲- بارگزاری دیتاست
۹	۱-۳- پیش پردازش و اصلاح مجموعه داده
۱۰	۱-۴- انتخاب دیتاست دیگر
۱۰	۱-۵- انجام بارگزاری و پیش پردازش بر دیتاست انتخابی جدید
۳۳	فصل ۲: تحلیل ویژگی های مدل
۳۴	۲-۱- تقسیم داده ها به آموزشی و تست
۳۴	۲-۲- بررسی ویژگی ها
۳۹	فصل ۳: آموزش و تست مدل
۵۳	فصل ۴: مراجع

فهرست اشکال

- شکل (۱-۱) بارگزاری دیتاست ۸
- شکل (2-1) خواندن دیتاست با کتابخانه pandas ۹
- شکل (۳-۱) زدایش ستون یکم مجموعه داده ۹
- شکل (۴-۱) بارگزاری دیتاست افسردگی دانش آموزان ۱۰
- شکل (5-1) حذف ستون اول به نام id ۱۱
- شکل (۶-۱) نمایش ستون سن ۱۱
- شکل (۷-۱) توزیع مقادیر سن ۱۲
- شکل (۸-۱) حذف دیتاهای خاص و پرت ۱۳
- شکل (۹-۱) گسسته سازی سن ۱۴
- شکل (۱۰-۱) مقادیر خاص ستون شهر ۱۵
- شکل (۱۱-۱) حذف مقادیر در شهرهای کم ۱۶
- شکل (۱۲-۱) عددی کردن مقادیر ستون شهر ۱۷
- شکل (۱۳-۱) تعداد مشاغل مختلف دیتاسمپل ها ۱۷
- شکل (۱۴-۱) مپ کردن شهرها به گروه های بزرگتر ۱۸
- شکل (۱۵-۱) مپ کردن سن به گروه های بزرگتر ۱۸
- شکل (۱۶-۱) تعداد گروه های فشار آکادمیک ۱۹
- شکل (۱۷-۱) پیاده سازی گروه بندی مجدد فشار آکادمیک ۱۹
- شکل (۱۸-۱) نتیجه گروه بندی مجدد فشار آکادمیک ۲۰
- شکل (۱۹-۱) گروه های ستون فشار کاری ۲۰
- شکل (20-1) مقادیر پیوسته CGPA ۲۱
- شکل (21-1) عددی سازی به ۱۰ گروه ستون CGPA ۲۲
- شکل (22-1) حذف ستون CGPA ۲۲
- شکل (۲۳-۱) وضعیت گروه های رضایت تحصیلی ۲۳
- شکل (۲۴-۱) مپینگ ستون رضایت تحصیلی ۲۳
- شکل (۲۵-۱) رضایت شغلی ۲۴
- شکل (۲۶-۱) آمار ستون مدت خواب ۲۴
- شکل (27-1) حذف دیتا سمپل های حاوی Others ۲۵
- شکل (۲۸-۱) مپ کردن مدت خواب ۲۵
- شکل (۲۹-۱) وضعیت ستون عادت غذایی ۲۶

فهرست اشکال

شکل (۱-۳۰) مپینگ عادت های غذایی	۲۶
شکل (۱-۳۱) ستون مدرک	۲۷
شکل (۱-۳۲) ستون مدرک پس از مپینگ	۲۸
شکل (۱-۳۳) ستون سابقه تفکر خودکشی	۲۸
شکل (1-34) مپینگ و دیجیتال سازی ستون سابقه تفکر خودکشی	۲۹
شکل (۱-۳۵) وضعیت گروه بندی ساعات مطالعه یا کار	۳۰
شکل (۱-۳۶) وضعیت ستون استرس اقتصادی	۳۱
شکل (۱-۳۷) وضعیت ستون سابقه بیماری روانی	۳۱
شکل (۱-۳۸) عددی سازی ستون سوابق فامیلی بیماری روانی	۳۲
شکل (۲-۱) تقسیم داده ها	۳۴
شکل (۲-۲) کد بررسی افسردگی براساس ویژگی های دیتاست	۳۴
شکل (۲-۳) بررسی درصد افسردگی براساس ویژگی سن	۳۴
شکل (۲-۴) بررسی درصد افسردگی براساس ویژگی سن	۳۵
شکل (۲-۵) بررسی درصد افسردگی براساس ویژگی شهر	۳۵
شکل (۲-۶) بررسی درصد افسردگی براساس ویژگی های فشار آکادمیک، رضایت تحصیلی، مدت خواب، عادت غذایی	۳۶
شکل (۲-۷) بررسی درصد افسردگی براساس ویژگی مدرک تحصیلی	۳۷
شکل (۲-۸) بررسی درصد افسردگی براساس ویژگی های سابقه افکار خودکشی، ساعت کار یا مطالعه، فشار اقتصادی، سابقه بیماری روانی خانواده و گروه CGPA	۳۸
شکل (۳-۱) درخت با حداکثر عمق ۱۷ و حداقل جداسازی ۱۰	۴۰
شکل (۳-۲) کاهش عمق درخت	۴۰
شکل (۳-۳) افزایش حداقل نمونه جداسازی به ۲۰	۴۱
شکل (۳-۴) افزایش حداقل نمونه جداسازی به ۳۰	۴۱
شکل (۳-۵) افزایش حداقل نمونه جداسازی به ۵۰	۴۲
شکل (۳-۶) افزایش حداقل نمونه جداسازی به ۱۰۰	۴۲
شکل (۳-۷) افزایش داده های یادگیری مدل	۴۳
شکل (۳-۸) کاهش عمق درخت	۴۳
شکل (۳-۹) افزایش داده های یادگیری	۴۴
شکل (۳-۱۰) افزایش عمق درخت به ۱۷	۴۴

فهرست اشکال

شکل (۱۱-۳) افزایش نسبت داده ی تست به یادگیری ۰.۲	۴۵
شکل (۱۲-۳) افزایش نسبت داده ی تست به یادگیری ۰.۳	۴۵
شکل (۱۳-۳) کاهش نسبت داده ی تست به ترین به ۰.۲۵	۴۶
شکل (۱۴-۳) افزایش حداقل سمپل جداسازی به ۱۱۰	۴۶
شکل (۱۵-۳) افزایش حداقل جداسازی نمونه به ۱۲۰	۴۷
شکل (۱۶-۳) افزایش حداقل جداسازی نمونه به ۱۳۰	۴۷
شکل (۱۷-۳) افزایش جداسازی نمونه به ۱۵۰	۴۸
شکل (۱۸-۳) افزایش جداسازی نمونه به ۲۰۰	۴۸
شکل (۱۹-۳) حذف مپینگ شهر	۴۸
شکل (۲۰-۳) تاثیر حذف مپینگ شهرها	۴۹
شکل (۲۱-۳) حذف مپینگ سن	۴۹
شکل (۲۲-۳) نتیجه حذف مپینگ سن	۵۰
شکل (23-3) افزایش مقادیر و بازه های ستون CGPA	۵۰
شکل (24-3) نتیجه ی افزایش بازه های ستون CGPA	۵۰
شکل (۲۵-۳) باز تعریف گروه های سنی دوباره به بازه های کوچک تر	۵۱
شکل (۲۶-۳) نتیجه و دقت پس از بازتعریف دوباره ی گروه های سنی	۵۱
شکل (۲۷-۳) افزایش بازه های سنی	۵۲
شکل (۲۸-۳) نتیجه ی افزایش بازه های سنی	۵۲
شکل (29-3) چاپ درخت تصمیم	۵۲

فصل ۱:

انتخاب و اصلاح اولیه دیتاست

۱-۱- انتخاب دیتاست

برای انتخاب یک دیتاست مناسب ابتدا به سایت Kaggle.com مراجعه کرده ام و پس از جست و جوی فراوان بین دیتاست های موجود در این وبسایت مجموعه داده ای به اسم "Credit Card Transactions Dataset" را به عنوان دیتاست این آموزش برگزیدم چرا که تعداد ستون های فراوانی به اندازه ۲۴ عدد داشت همچنین اکثر این ستون ها به نظر کاربردی بودند و تعداد نمونه داده هایی که در این مجموعه ی داده وجود دارد به اندازه حدود ۱ میلیون سطر است که عدد قابل توجه و زیادی هست و با توجه به این میزان داده ای که در اختیار داریم و تعداد ویژگی های زیادی که این مجموعه داده در اختیار ما قرار می دهد انتظار می رود بتوان یک مدل خوب برای آن پیدا کرد.

۱-۲- بارگزاری دیتاست

به سایت کولب مراجعه کرده و پروژه جدید ایجاد می کنیم سپس دیتاست خود را بر وبسایت و سرور های کولب بارگزاری می کنیم.

```

] !pip install kagglehub

import kagglehub
import pandas as pd
import shutil

target_path = "/content/credit_card_dataset"

path = kagglehub.dataset_download("priyamchoksi/credit-card-transactions-dataset")

shutil.move(path, target_path)

print("Dataset stored at:", target_path)

```

شکل (۱-۱) بارگزاری دیتاست

پس از این مرحله بارگزاری دیتاست انجام شده و به ما محل ذخیره فایل مجموعه داده چاپ می شود حالا وقت آن است که با استفاده از کتابخانه pandas این دیتاست را به پایتون بیاوریم.


```
full_data = pd.read_csv("/content/credit_card_dataset/credit_card_transactions.csv")
full_data
```

	Unnamed: 0	trans_date_trans_time	cc_num	merchant	category	amt	first	last	gender	street	..
0	0	2019-01-01 00:00:18	2703186189652095	fraud_Rippin, Kub and Mann	misc_net	4.97	Jennifer	Banks	F	561 Perry Cove	..
1	1	2019-01-01 00:00:44	630423337322	fraud_Heller, Gutmann and Zieme	grocery_pos	107.23	Stephanie	Gill	F	43039 Riley Greens Suite 393	..
2	2	2019-01-01 00:00:51	38859492057661	fraud_Lind-Buckridge	entertainment	220.11	Edward	Sanchez	M	594 White Dale Suite 530	..
3	3	2019-01-01 00:01:16	3534093764340240	fraud_Kutch, Hermiston and Farrell	gas_transport	45.00	Jeremy	White	M	9443 Cynthia Court Apt. 038	..
4	4	2019-01-01 00:03:06	375534208663984	fraud_Keeling-Crist	misc_pos	41.96	Tyler	Garcia	M	408 Bradley Rest	..
...	...	...	...	...	...	...	...	...	...	...	..

شکل (۱-۲) خواندن دیتاست با کتابخانه pandas

۱-۳- پیش پردازش و اصلاح مجموعه داده

همانطور که از شکل (۱-۲) واضح است این دیتاست ستون هایی دارد که ممکن است در آینده برای ما دردسر ساز شوند برای مثال ستون های ردیف که مشخص است می توانند باعث بیش برازش مدل ما شوند را از همین حالا از مجموعه داده ی خود بر می داریم.

```
# Let's drop first column which is not a good attribute for us
full_data = full_data.drop(columns = full_data.columns[0])
full_data
```

	trans_date_trans_time	cc_num	merchant	category	amt	first	last	gender	street	city
0	2019-01-01 00:00:18	2703186189652095	fraud_Rippin, Kub and Mann	misc_net	4.97	Jennifer	Banks	F	561 Perry Cove	Moravian Falls
1	2019-01-01 00:00:44	630423337322	fraud_Heller, Gutmann and Zieme	grocery_pos	107.23	Stephanie	Gill	F	43039 Riley Greens Suite 393	Orient
2	2019-01-01 00:00:51	38859492057661	fraud_Lind-Buckridge	entertainment	220.11	Edward	Sanchez	M	594 White Dale Suite 530	Malad City
3	2019-01-01 00:01:16	3534093764340240	fraud_Kutch, Hermiston and Farrell	gas_transport	45.00	Jeremy	White	M	9443 Cynthia Court Apt. 038	Boulder
4	2019-01-01 00:03:06	375534208663984	fraud_Keeling-Crist	misc_pos	41.96	Tyler	Garcia	M	408 Bradley Rest	Doe Hill

شکل (۱-۳) زدایش ستون یکم مجموعه داده

دوباره یک ستون دیگر هم داریم و آن را هم می زداییم.

۴-۱- انتخاب دیتاست دیگر

به سبب آنکه این دیتاست ممکن بود با درخت تصمیم به نتیجه نرسد دیتاست دیگری را در وبسایت Kaggle.com پیدا کردم با عنوان "Student Depression Dataset" که با ۱۸ ستون دیتاست بهتری به شمار می رود همچنین تعداد دیتا سمپل های آن هم ۲۷۹۰۰ است در نتیجه همین دیتاست را انتخاب می کنیم چون به نظر بتوان در یک درخت تصمیم آن را پیاده سازی کرد.

۵-۱- انجام بارگزاری و پیش پردازش بر دیتاست انتخابی جدید

برای دیتاست جدید هم مراحل قبل را انجام می دهیم:

```
[9] import shutil
shutil.unpack_archive("/content/archive.zip", "/content")

df = pd.read_csv("/content/Student Depression Dataset.csv")
df
```

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts
0	2	Male	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Pharm	Yes
1	8	Female	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	BSc	No
2	26	Male	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Healthy	BA	No
3	30	Female	28.0	Varanasi	Student	3.0	0.0	5.59	2.0	0.0	7-8 hours	Moderate	BCA	Yes
4	32	Female	25.0	Jaipur	Student	4.0	0.0	8.13	3.0	0.0	5-6 hours	Moderate	M.Tech	Yes
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
27896	140685	Female	27.0	Surat	Student	5.0	0.0	5.75	5.0	0.0	5-6 hours	Unhealthy	Class 12	Yes

شکل (۴-۱) بارگزاری دیتاست افسردگی دانش آموزان

حالا زمان آن است که دیتاست را اصلاح کنیم بنابراین از ستون آیدی شروع می کنیم و آن را حذف می

کنیم.

```
[15] #remove id column
df = df.drop(columns = df.columns[0])
```

```
[16] df
```

	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Work/s F
0	Male	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Pharm	Yes	
1	Female	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	BSc	No	
2	Male	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Healthy	BA	No	
3	Female	28.0	Varanasi	Student	3.0	0.0	5.59	2.0	0.0	7-8 hours	Moderate	BCA	Yes	
4	Female	25.0	Jaipur	Student	4.0	0.0	8.13	3.0	0.0	5-6 hours	Moderate	M.Tech	Yes	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
27896	Female	27.0	Surat	Student	5.0	0.0	5.75	5.0	0.0	5-6 hours	Unhealthy	Class 12	Yes	
27897	Male	27.0	Ludhiana	Student	2.0	0.0	9.40	3.0	0.0	Less than 5 hours	Healthy	MSc	No	

شکل (۵-۱) حذف ستون اول به نام id

میخواهیم به چندین گروه سن را تقسیم کنیم اما بهتر است اول مینیمم و ماکزیمم آن را داشته باشیم.

```
#let's map ages to discrete values
print(df['Age'])
#df['Age_Category'] = pd.qcut(df['Age'])
```

0	33.0
1	24.0
2	31.0
3	28.0
4	25.0
...	...
27896	27.0
27897	27.0
27898	31.0
27899	18.0
27900	27.0

Name: Age, Length: 27901, dtype: float64

شکل (۶-۱) نمایش ستون سن

پس به سراغ تحلیل داده می رویم:

value	counts:Age
24.0	2258
20.0	2237
28.0	2133
29.0	1950
33.0	1893
25.0	1784
21.0	1726
23.0	1645
18.0	1587
19.0	1560
34.0	1468
27.0	1462
31.0	1427
32.0	1262
22.0	1160
26.0	1155
30.0	1145
35.0	10
38.0	8
36.0	7
42.0	4
39.0	3
48.0	3
46.0	2
43.0	2
37.0	2
44.0	1
51.0	1
49.0	1
58.0	1
54.0	1
59.0	1
56.0	1
41.0	1

شکل (۷-۱) توزیع مقادیر سن

در شکل ۷-۱ مشاهده می شود که از سن ۳۴ به بعد داده های ما زیاد نیستند در نتیجه برای اینکه از بیش برآزش مدل جلوگیری شود این دیتا ها را حذف می کنیم.

```
import pandas as pd
#remove datasamples with ages more than 34
#type(df)
#df = df.to_frame()
#print(df.columns)
df = df[df['Age']<=34]

print(f"unique values: {df['Age'].nunique()}")
print(f"value counts:{df['Age'].value_counts()}")
```

```
unique values: 17
value counts:Age
24.0    2258
20.0    2237
28.0    2133
29.0    1950
33.0    1893
25.0    1784
21.0    1726
23.0    1645
18.0    1587
19.0    1560
34.0    1468
27.0    1462
31.0    1427
32.0    1262
22.0    1160
26.0    1155
30.0    1145
Name: count, dtype: int64
```

شکل (۸-۱) حذف دیتا های خاص و پرت

حالا وقت آن است که مپینگ دیتا ها را انجام دهیم اما با دقت بیشتری بر مدل در میابیم که تقریباً دیتا ها در گروه های خاص و سن های خاصی هستند در نتیجه مپینگ را انجام نمی دهیم.

```
df['Gender'] = df['Gender'].map({df.loc[0]['Gender']: 0, df.loc[1]['Gender']: 1}).astype(float)
```

<ipython-input-105-45ec618216dc>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df['Gender'] = df['Gender'].map({df.loc[0]['Gender']: 0, df.loc[1]['Gender']: 1}).astype(float)

df

	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Wo
0	0.0	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Pharm	Yes	
1	1.0	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	BSc	No	
2	0.0	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Healthy	BA	No	
3	1.0	28.0	Varanasi	Student	3.0	0.0	5.59	2.0	0.0	7-8 hours	Moderate	BCA	Yes	
4	1.0	25.0	Jaipur	Student	4.0	0.0	8.13	3.0	0.0	5-6 hours	Moderate	M.Tech	Yes	

شکل (۹-۱) گسسته سازی سن

Patna	1003
Pune	967
Visakhapatnam	966
Ahmedabad	951
Bhopal	932
Chennai	883
Meerut	823
Rajkot	813
Delhi	767
Bangalore	766
Ghaziabad	743
Mumbai	698
Vadodara	693
Varanasi	684
Nagpur	650
Indore	643
Kanpur	609
Nashik	545
Faridabad	461
Saanvi	2
Bhavna	2
City	2
Harsha	2
Less Delhi	1
M.Tech	1
3.0	1
Less than 5 Kalyan	1
Mira	1
Vaanya	1
Gaurav	1
Harsh	1
Reyansh	1
Kibara	1
Rashi	1
ME	1

شکل (۱۰-۱) مقادیر خاص ستون شهر

```
for city in cities_index:
    df = df[df['City']!=city]

print(f"nunique:{df['City'].nunique()}")
print(f"count_values:{df['City'].value_counts()}")
```

```
nunique:30
count_values:City
Kalyan          1567
Srinagar        1370
Hyderabad       1338
Vasai-Virar     1289
Lucknow         1153
Thane           1138
Ludhiana        1108
Agra            1091
Surat           1076
Kolkata         1063
Jaipur          1036
Patna           1003
Pune            967
Visakhapatnam   966
Ahmedabad       951
Bhopal          932
Chennai         883
Meerut          823
Rajkot          813
Delhi           767
Bangalore       766
Ghaziabad       743
Mumbai          698
Vadodara        693
```

شکل (۱-۱۱) حذف مقادیر در شهر های کم

به این دلیل که اگر یک شهر خاصی برای مثال یک دانش آموز افسرده داشته باشد ممکن است مدل ما به اشتباه این تصمیم را بگیرد که این شهر هرچی دانش آموز دارد افسرده است در نتیجه این ها را هم حذف کرده ایم.


```
city_mapping = {city: i for i, city in enumerate(remained_cities)}
```

```
filterdf = df.copy()
filterdf['City'] = filterdf['City'].map(city_mapping)
```

```
filterdf
```

	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Work/Study Hours
0	0.0	33.0	13	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Pharm	Yes	3.0
1	1.0	24.0	20	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	BSc	No	3.0
2	0.0	31.0	1	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Healthy	BA	No	9.0
3	1.0	28.0	24	Student	3.0	0.0	5.59	2.0	0.0	7-8 hours	Moderate	BCA	Yes	4.0
4	1.0	25.0	10	Student	4.0	0.0	8.13	3.0	0.0	5-6 hours	Moderate	M.Tech	Yes	1.0

شکل (۱۲) عددی کردن مقادیر ستون شهر

```
# Check Profession
print(df['Profession'].value_counts())
```

```
Profession
Student                27795
Architect                8
Teacher                 6
Digital Marketer        3
Chef                    2
Content Writer          2
Pharmacist              2
Doctor                  2
UX/UI Designer          1
Civil Engineer          1
Manager                 1
Educational Consultant  1
Lawyer                  1
Entrepreneur            1
Name: count, dtype: int64
```

شکل (۱۳) تعداد مشاغل مختلف دیتاسمپل ها

با توجه به اینکه این ستون حدود ۹۹ درصد آن یک مقدار دانش آموز را دارد بهتر است آن را حذف کنیم تا از کاهش برازش و underfitting جلوگیری کنیم.

```
# city mapping
filterdf['City'] = filterdf['City'].fillna(filterdf['City'].median())
filterdf.loc[filterdf['City'] <= 2, 'City'] = 0
filterdf.loc[(filterdf['City'] > 2) & (filterdf['City'] <= 6), 'City'] = 1
filterdf.loc[(filterdf['City'] > 6) & (filterdf['City'] <= 11), 'City'] = 2
filterdf.loc[(filterdf['City'] > 11) & (filterdf['City'] <= 16), 'City'] = 3
filterdf.loc[(filterdf['City'] > 16) & (filterdf['City'] <= 21), 'City'] = 4
filterdf.loc[(filterdf['City'] > 21) & (filterdf['City'] <= 29), 'City'] = 5
```

شکل (۱-۱۴) مپ کردن شهر ها به گروه های بزرگتر

```
# Age mapping
filterdf['Age'] = filterdf['Age'].fillna(filterdf['Age'].median())
filterdf.loc[filterdf['Age'] <= 20, 'Age'] = 0
filterdf.loc[(filterdf['Age'] > 20) & (filterdf['Age'] <= 23), 'Age'] = 1
filterdf.loc[(filterdf['Age'] > 23) & (filterdf['Age'] <= 25), 'Age'] = 2
filterdf.loc[(filterdf['Age'] > 25) & (filterdf['Age'] <= 28), 'Age'] = 3
filterdf.loc[(filterdf['Age'] > 28) & (filterdf['Age'] <= 31), 'Age'] = 4
filterdf.loc[(filterdf['Age'] > 31) & (filterdf['Age'] <= 34), 'Age'] = 5
filterdf
```

شکل (۱-۱۵) مپ کردن سن به گروه های بزرگتر

```
print(df['Academic Pressure'].nunique())
print(df['Academic Pressure'].value_counts())
```

✓ 0.0s

```
6
Academic Pressure
3.0    7443
5.0    6278
4.0    5147
1.0    4784
2.0    4167
0.0       7
Name: count, dtype: int64
```

شکل (۱-۱۶) تعداد گروه های فشار آکادمیک

در اینجا گروه صفر خیلی تعدادش کمه پس با گروه ۱ یکی می کنیم.

```
finaldf = df.copy()
filterdf = df.copy()
nameToFilter = 'Academic Pressure'
# Age mapping
filterdf[nameToFilter] = filterdf[nameToFilter].fillna(filterdf[nameToFilter].median())
filterdf.loc[filterdf[nameToFilter] <= 1, nameToFilter] = 0
filterdf.loc[(filterdf[nameToFilter] > 1) & (filterdf[nameToFilter] <= 2), nameToFilter] = 1
filterdf.loc[(filterdf[nameToFilter] > 2) & (filterdf[nameToFilter] <= 3), nameToFilter] = 2
filterdf.loc[(filterdf[nameToFilter] > 3) & (filterdf[nameToFilter] <= 4), nameToFilter] = 3
filterdf.loc[(filterdf[nameToFilter] > 4) & (filterdf[nameToFilter] <= 5), nameToFilter] = 4
filterdf
```

شکل (۱-۱۷) پیاده سازی گروه بندی مجدد فشار آکادمیک

```
df = filterdf.copy()
finaldf = df.copy()

print(df[nameToFilter].nunique())
print(df[nameToFilter].value_counts())
```

✓ 0.0s

```
5
Academic Pressure
2.0    7443
4.0    6278
3.0    5147
0.0    4791
1.0    4167
Name: count, dtype: int64
```

شکل (۱-۱۸) نتیجه گروه بندی مجدد فشار آکادمیک

به سراغ فشار کاری می رویم.

```
filterdf = df.copy()
nameToFilter = 'Work Pressure'
print(filterdf[nameToFilter].nunique())
print(filterdf[nameToFilter].value_counts())
```

✓ 0.0s

```
3
Work Pressure
0.0    27824
2.0         1
5.0         1
Name: count, dtype: int64
```

شکل (۱-۱۹) گروه های ستون فشار کاری

مشاهده می شود که این ستون هم بی تاثیر است و باید حذف گردد.

می رویم به سراغ ستون CGPA:

```
df = filterdf.copy()
finaldf = df.copy()

# CGPA
nameToFilter = 'CGPA'
print(filterdf[nameToFilter].nunique())
print(filterdf[nameToFilter].value_counts())
filterdf
```

✓ 0.0s

```
331
CGPA
8.0400      821
9.9600      424
5.7400      409
8.9500      370
9.2100      341
...
6.6400        1
7.0625        1
6.9800        1
6.4400        1
6.0900        1
```

شکل (۲۰-۱) مقادیر پیوسته CGPA

```
# Discretize CGPA into 10 groups
filterdf['CGPA_Group'] = pd.qcut(filterdf[nameToFilter], q=10, labels=[0,1,2,3,4,5,6,7,8,9])

print("Distribution of CGPA groups:")
print(filterdf['CGPA_Group'].value_counts().sort_index())
```

✓ 0.0s

Distribution of CGPA groups:

CGPA_Group

0	2844
1	2816
2	2801
3	2694
4	2869
5	2867
6	2625
7	2793
8	2766
9	2751

شکل (۱-۲۱) عددی سازی به ۱۰ گروه ستون CGPA

```
# drop CGPA
filterdf = filterdf.drop(columns=nameToFilter)
filterdf
```

✓ 0.0s

شکل (۱-۲۲) حذف ستون CGPA

```

nameToFilter = 'Study Satisfaction'
print(filterdf[nameToFilter].unique())
print(filterdf[nameToFilter].value_counts())

```

✓ 0.0s

6

Study Satisfaction

4.0	6347
2.0	5825
3.0	5805
1.0	5431
5.0	4409
0.0	9

شکل (۲۳-۱) وضعیت گروه های رضایت تحصیلی

باید گروه صفر و یک باهم یکی شوند.

```

# Age mapping
filterdf[nameToFilter] = filterdf[nameToFilter].fillna(filterdf[nameToFilter].median())
filterdf.loc[filterdf[nameToFilter] <= 1, nameToFilter] = 0
filterdf.loc[(filterdf[nameToFilter] > 1) & (filterdf[nameToFilter] <= 2), nameToFilter] = 1
filterdf.loc[(filterdf[nameToFilter] > 2) & (filterdf[nameToFilter] <= 3), nameToFilter] = 2
filterdf.loc[(filterdf[nameToFilter] > 3) & (filterdf[nameToFilter] <= 4), nameToFilter] = 3
filterdf.loc[(filterdf[nameToFilter] > 4) & (filterdf[nameToFilter] <= 5), nameToFilter] = 4
print(filterdf[nameToFilter].unique())
print(filterdf[nameToFilter].value_counts())

```

✓ 0.0s

5

Study Satisfaction

3.0	6347
1.0	5825
2.0	5805
0.0	5440
4.0	4409

Name: count, dtype: int64

شکل (۲۴-۱) مپینگ ستون رضایت تحصیلی

```

nameToFilter = 'Job Satisfaction'
print(filterdf[nameToFilter].nunique())
print(filterdf[nameToFilter].value_counts())

```

✓ 0.0s

5

Job Satisfaction	
0.0	27819
2.0	3
1.0	2
3.0	1
4.0	1

شکل (۱-۲۵) رضایت شغلی

حذف این ستون هم واضح است که ضروری است.

```

df = filterdf.copy()
finaldf = df.copy()

```

Ctrl+I to chat, Ctrl+K to generate

```

nameToFilter = 'Sleep Duration'
print(filterdf[nameToFilter].nunique())
print(filterdf[nameToFilter].value_counts())

```

✓ 0.0s

5

Sleep Duration	
Less than 5 hours	8294
7-8 hours	7324
5-6 hours	6159
More than 8 hours	6031
Others	18

شکل (۱-۲۶) آمار ستون مدت خواب

باید ستون سایر را با ستون بیش از ۸ ساعت یکی کنیم وگرنه ممکن است بیش برآزش رخ دهد البته به این دلیل که نمی دانیم جزو کدام گروه هستند و ممکن است مدل گیج شود و به اشتباه بی افتد در نتیجه کلاً این دیتا سمپل ها را حذف می کنیم.

```
# Remove rows where Sleep Duration is 'Others'
filterdf = filterdf[filterdf['Sleep Duration'] != 'Others']

print(filterdf['Sleep Duration'].value_counts())
```

✓ 0.0s

Sleep Duration	
Less than 5 hours	8294
7-8 hours	7324
5-6 hours	6159
More than 8 hours	6031
Name: count, dtype: int64	

شکل (۱-۲۷) حذف دیتا سمپل های حاوی Others

```
filterdf[nameToFilter] = filterdf[nameToFilter].map({"Less than 5 hours":0, "5-6 hours":1, "7-8 hours":2, "More than 8 hours":3})

print(filterdf[nameToFilter].nunique())
print(filterdf[nameToFilter].value_counts())
```

✓ 0.0s Python

C:\Users\Amir\AppData\Local\Temp\ipykernel_16184\1462080331.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
filterdf[nameToFilter] = filterdf[nameToFilter].map({"Less than 5 hours":0, "5-6 hours":1, "7-8 hours":2, "More than 8 hours":3})
```

Sleep Duration	
0	8294
2	7324
1	6159
3	6031

شکل (۱-۲۸) مپ کردن مدت خواب

```
nameToFilter = 'Dietary Habits'
print(filterdf[nameToFilter].nunique())
print(filterdf[nameToFilter].value_counts())
```

✓ 0.0s

```
4
Dietary Habits
Unhealthy    10284
Moderate      9885
Healthy       7627
Others         12
```

شکل (۱-۲۹) وضعیت ستون عادت غذایی

همانطور که در شکل ۱-۲۹ واضح است باید گزینه سایر را دوباره حذف کنیم تا بیش برآزش رخ ندهد.

```
filterdf = filterdf[filterdf[nameToFilter]!='Others']
filterdf[nameToFilter] = filterdf[nameToFilter].map({"Unhealthy":0, "Moderate":1, "Healthy":2})
filterdf
```

✓ 0.0s

	Gender	Age	City	Academic Pressure	Study Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Work/Study Hours	Financial Stress	Family History of Mental Illness	Depression
0	0.0	5.0	3	4.0	1.0	1	2	B.Pharm	Yes	3.0	1.0	No	1
1	1.0	2.0	4	1.0	4.0	1	1	BSc	No	3.0	2.0	Yes	0
2	0.0	4.0	0	2.0	4.0	0	2	BA	No	9.0	1.0	Yes	0
3	1.0	3.0	5	2.0	1.0	2	1	BCA	Yes	4.0	5.0	Yes	1
4	1.0	2.0	2	3.0	2.0	1	1	M.Tech	Yes	1.0	1.0	No	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
7896	1.0	3.0	2	4.0	4.0	1	0	Class 12	Yes	7.0	1.0	Yes	0
7897	0.0	3.0	1	1.0	2.0	0	2	MSc	No	0.0	3.0	Yes	0
7898	0.0	4.0	5	2.0	3.0	1	0	MD	No	12.0	2.0	No	0
7899	1.0	0.0	1	4.0	1.0	0	2	Class 12	Yes	10.0	5.0	No	1
7900	0.0	3.0	2	3.0	0.0	0	2	BCA	Yes	2.0	3.0	Yes	1

شکل (۱-۳۰) مپینگ عادت های غذایی

```
nameToFilter = 'Degree'
print(filterdf[nameToFilter].nunique())
print(filterdf[nameToFilter].value_counts())
```

✓ 0.0s

28

Degree

Class 12	6062
B.Ed	1860
B.Com	1501
B.Arch	1474
BCA	1430
MSc	1185
B.Tech	1151
MCA	1038
M.Tech	1018
BHM	923
BSc	882
M.Ed	815
B.Pharm	809
M.Com	731
BBA	696
MBBS	690
LLB	668
BE	609
BA	595
M.Pharm	580
MD	569
MBA	560
MA	544

شکل (۳۱-۱) ستون مدرک

```
# Map cities to digital values
degree_mapping = {city: i for i, city in enumerate(degree_list)}

filterdf = filterdf.copy()
filterdf[nameToFilter] = filterdf[nameToFilter].map(degree_mapping)

filterdf
0.0s
```

	Gender	Age	City	Academic Pressure	Study Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Work/Study Hours	Financial Stress	Family History of Mental Illness	Depression
0	0.0	5.0	3	4.0	1.0	1	2	12	Yes	3.0	1.0	No	1
1	1.0	2.0	4	1.0	4.0	1	1	10	No	3.0	2.0	Yes	0
2	0.0	4.0	0	2.0	4.0	0	2	18	No	9.0	1.0	Yes	0
3	1.0	3.0	5	2.0	1.0	2	1	4	Yes	4.0	5.0	Yes	1
4	1.0	2.0	2	3.0	2.0	1	1	8	Yes	1.0	1.0	No	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
7896	1.0	3.0	2	4.0	4.0	1	0	0	Yes	7.0	1.0	Yes	0
7897	0.0	3.0	1	1.0	2.0	0	2	5	No	0.0	3.0	Yes	0
7898	0.0	4.0	5	2.0	3.0	1	0	20	No	12.0	2.0	No	0
7899	1.0	0.0	1	4.0	1.0	0	2	0	Yes	10.0	5.0	No	1
7900	0.0	3.0	2	3.0	0.0	0	2	4	Yes	2.0	3.0	Yes	1

شکل (۱-۳۲) ستون مدرک پس از مپینگ

```
nameToFilter = 'Have you ever had suicidal thoughts ?'
print(filterdf[nameToFilter].unique())
print(filterdf[nameToFilter].value_counts())
```

✓ 0.0s

```
2
Have you ever had suicidal thoughts ?
Yes      17601
No       10195
```

شکل (۱-۳۳) ستون سابقه تفکر خودکشی

```
filterdf[nameToFilter] = filterdf[nameToFilter].map({'Yes':1, 'No':0}).astype(float)
✓ 0.0s

nameToFilter = 'Have you ever had suicidal thoughts ?'
print(filterdf[nameToFilter].nunique())
print(filterdf[nameToFilter].value_counts())
df = filterdf.copy()
finaldf = df.copy()
✓ 0.0s

2
Have you ever had suicidal thoughts ?
1.0      17601
0.0      10195
```

شکل (۱-۳۴) مپینگ و دیجیتال سازی ستون سابقه تفکر خودکشی

```

nameToFilter = 'Work/Study Hours'
print(filterdf[nameToFilter].nunique())
print(filterdf[nameToFilter].value_counts())
✓ 0.0s

```

13	
Work/Study Hours	
10.0	4223
12.0	3162
11.0	2884
8.0	2502
6.0	2239
9.0	2019
7.0	1992
0.0	1692
4.0	1604
2.0	1578
3.0	1464
5.0	1293
1.0	1144

شکل (۳۵-۱) وضعیت گروه بندی ساعات مطالعه یا کار

به نظر مشکل خاصی وجود نداره و تفکیک به خوبی صورت می گیرد اگر مشکلی پیش بیاید می توانیم گروه ها را کوچک تر کنیم تا از بیش برآزش جلوگیری شود.

```

nameToFilter = fd.columns[10]
print(filterdf[nameToFilter].nunique())
print(filterdf[nameToFilter].value_counts())

```

✓ 0.0s

5

Financial Stress

5.0	6688
4.0	5753
3.0	5211
1.0	5101
2.0	5040

Name: count, dtype: int64

شکل (۱-۳۶) وضعیت ستون استرس اقتصادی

```

nameToFilter = df.columns[11]
print(df[nameToFilter].nunique())
print(df[nameToFilter].value_counts())

```

✓ 0.0s

2

Family History of Mental Illness

No	14345
Yes	13451

شکل (۱-۳۷) وضعیت ستون سابقه بیماری روانی

این ستون را هم باید دیجیتال کنیم.

```
filterdf[nameToFilter] = filterdf[nameToFilter].map({'Yes':1, 'No':0})
filterdf
```

✓ 0.0s

	Gender	Age	City	Academic Pressure	Study Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Work/Study Hours	Financial Stress	Family History of Mental Illness	Depression	C
0	0.0	5.0	3	4.0	1.0	1	2	12	1.0	3.0	1.0	0	1	
1	1.0	2.0	4	1.0	4.0	1	1	10	0.0	3.0	2.0	1	0	
2	0.0	4.0	0	2.0	4.0	0	2	18	0.0	9.0	1.0	1	0	
3	1.0	3.0	5	2.0	1.0	2	1	4	1.0	4.0	5.0	1	1	
4	1.0	2.0	2	3.0	2.0	1	1	8	1.0	1.0	1.0	0	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
27896	1.0	3.0	2	4.0	4.0	1	0	0	1.0	7.0	1.0	1	0	
27897	0.0	3.0	1	1.0	2.0	0	2	5	0.0	0.0	3.0	1	0	
27898	0.0	4.0	5	2.0	3.0	1	0	20	0.0	12.0	2.0	0	0	
27899	1.0	0.0	1	4.0	1.0	0	2	0	1.0	10.0	5.0	0	1	
27900	0.0	3.0	2	3.0	0.0	0	2	4	1.0	2.0	3.0	1	1	

شکل (۳۸-۱) عددی سازی ستون سوابق فامیلی بیماری روانی

خب و ستون افسردگی هم که ستون تارگت ما هست هم عددی است بنابراین پیش پردازش این دیتاست تکمیل شد.

فصل ۲: تحلیل ویژگی های مدل

۲-۱- تقسیم داده ها به آموزشی و تست

```
from sklearn.model_selection import train_test_split

X = full_data.drop(columns=[full_data_columns[0]]) # Features
y = full_data[full_data_columns[-2]] # Target

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=19457)

# Print sizes
print(len(X_train), len(X_test))
```

شکل (۲-۱) تقسیم داده ها

مقدار ۳۰ درصد داده ها را به داده های تست اختصاص داده و بقیه ۷۰ درصد را به داده های آموزشی مدل

۲-۲- بررسی ویژگی ها

```
for column in full_data_columns:
    if(column != full_data_columns[-2]):
        print(full_data[[column, full_data_columns[-2]]].groupby([column], as_index=False).mean())
```

شکل (۲-۲) کد بررسی افسردگی براساس ویژگی های دیتاست

	Gender	Depression
0	0.0	0.586732
1	1.0	0.585285

شکل (۲-۳) بررسی درصد افسردگی براساس ویژگی سن

طبق شکل ۳-۳ ویژگی سن زیاد ویژگی خوب و جدا کننده ای نیست اما در هر صورت شاید به کار ما بیاید.

	Age	Depression
0	0.0	0.723689
1	1.0	0.644612
2	2.0	0.641159
3	3.0	0.601477
4	4.0	0.500777
5	5.0	0.387782

شکل (۲-۴) بررسی درصد افسردگی براساس ویژگی سن

	City	Depression
0	0	0.606649
1	1	0.575699
2	2	0.575499
3	3	0.604473
4	4	0.604199
5	5	0.557878

شکل (۲-۵) بررسی درصد افسردگی براساس ویژگی شهر

Academic Pressure		Depression
0	0.0	0.194862
1	1.0	0.375000
2	2.0	0.602286
3	3.0	0.761868
4	4.0	0.861470
Study Satisfaction		Depression
0	0.0	0.708004
1	1.0	0.646331
2	2.0	0.577042
3	3.0	0.513253
4	4.0	0.472740
Sleep Duration		Depression
0	0	0.645554
1	1	0.569363
2	2	0.595820
3	3	0.509619
Dietary Habits		Depression
0	0	0.707896
1	1	0.561153
2	2	0.454176

شکل (۶-۲) بررسی درصد افسردگی براساس ویژگی های فشار آکادمیک، رضایت تحصیلی، مدت خواب، عادت غذایی

	Degree	Depression
0	0	0.708512
1	1	0.546237
2	2	0.566955
3	3	0.590231
4	4	0.571329
5	5	0.572152
6	6	0.568202
7	7	0.534682
8	8	0.508841
9	9	0.551463
10	10	0.590703
11	11	0.505521
12	12	0.529048
13	13	0.533516
14	14	0.584770
15	15	0.579710
16	16	0.531437
17	17	0.543514
18	18	0.536134
19	19	0.539655
20	20	0.523726
21	21	0.537500
22	22	0.533088
23	23	0.555340
24	24	0.538462
25	25	0.518325
26	26	0.532609
27	27	0.600000

شکل (۲-۷) بررسی درصد افسردگی براساس ویژگی مدرک تحصیلی

Have you ever had suicidal thoughts ?		Depression
0	0.0	0.232271
1	1.0	0.791035
Work/Study Hours		Depression
0	0.0	0.355792
1	1.0	0.403846
2	2.0	0.439163
3	3.0	0.475410
4	4.0	0.504988
5	5.0	0.546017
6	6.0	0.573917
7	7.0	0.587851
8	8.0	0.625100
9	9.0	0.600297
10	10.0	0.703765
11	11.0	0.678571
12	12.0	0.683428
Financial Stress		Depression
0	1.0	0.318761
1	2.0	0.430556
2	3.0	0.589906
3	4.0	0.692161
4	5.0	0.813098
Family History of Mental Illness		Depression
0	0	0.560613
1	1	0.613263
CGPA_Group		Depression
0	0	0.560014
1	1	0.555635

شکل (۸-۲) بررسی درصد افسردگی براساس ویژگی های سابقه افکار خودکشی، ساعت کار یا مطالعه، فشار اقتصادی، سابقه بیماری روانی خانواده و گروه CGPA

فصل ۳: آموزش و تست مدل

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=10, max_depth=17, random_state=42)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print("Train Accuracy:", train_accuracy)
print("Test Accuracy:", test_accuracy)
```

Train Accuracy: 0.9313871614329033
Test Accuracy: 0.777551265139705

شکل (۳-۱) درخت با حداکثر عمق ۱۷ و حداقل جداسازی ۱۰

مطابق شکل ۳-۹ و خروجی دقت آموزش مدل بسیار خوب است اما با نگاهی به داده ی تست در می یابیم که بیش برآزش رخ داد است به این سبب که دقت داده ی تست خیلی کمتر از داده ی آموزشی است بنابراین دلخواه ما این است که داده ی تست بهتر شود ولو به این قیمت که از دقت داده ی آموزشی کم شود و اختلاف را باید کاهش دهیم.

برای این که بیش برآزش را کاهش دهیم می توانیم عمق درخت را کم کنیم یا حداقل جداسازی درخت را بیشتر کنیم که با مورد اول شروع می کنیم.

```
classifier = DecisionTreeClassifier(min_samples_split=10, max_depth=15, random_state=42)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print("Train Accuracy:", train_accuracy)
print("Test Accuracy:", test_accuracy)
```

Train Accuracy: 0.9277894845042916
Test Accuracy: 0.7811488188032139

شکل (۳-۲) کاهش عمق درخت

مشاهده می شود که کمی نتیجه بهتر شد اما هنوز دلخواه ما نیست.


```

classifier = DecisionTreeClassifier(min_samples_split=20, max_depth=15, random_state=42)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print("Train Accuracy:", train_accuracy)
print("Test Accuracy:", test_accuracy)

```

Train Accuracy: 0.8948450429151462
Test Accuracy: 0.7997361793980093

شکل (۳-۳) افزایش حداقل نمونه جداسازی به ۲۰

```

classifier = DecisionTreeClassifier(min_samples_split=30, max_depth=15, random_state=42)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print("Train Accuracy:", train_accuracy)
print("Test Accuracy:", test_accuracy)

```

Train Accuracy: 0.87988898596906
Test Accuracy: 0.8066914498141264

شکل (۳-۴) افزایش حداقل نمونه جداسازی به ۳۰

دقت ترین و تست به هم نزدیک شدند اما باید توجه داشته باشیم که ما نمی خواهیم دقت یادگیری از ۸۵ کمتر شود.

```

classifier = DecisionTreeClassifier(min_samples_split=50, max_depth=15, random_state=42)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print("Train Accuracy:", train_accuracy)
print("Test Accuracy:", test_accuracy)

```

Train Accuracy: 0.8645217659454181
Test Accuracy: 0.817963784626454

شکل (۳-۵) افزایش حداقل نمونه جداسازی به ۵۰

```

classifier = DecisionTreeClassifier(min_samples_split=100, max_depth=15, random_state=42)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print("Train Accuracy:", train_accuracy)
print("Test Accuracy:", test_accuracy)

```

Train Accuracy: 0.8503366397697487
Test Accuracy: 0.8268377503297758

شکل (۳-۶) افزایش حداقل نمونه جداسازی به ۱۰۰

اینجا دیگر نمی‌توانیم بیشتر کنیم نمونه جداسازی را به این سبب که ما نیاز داریم که حداقل دقت ۸۵ را حفظ کنیم هرچند که این دقت یادگیری ما است. یک بار تقسیم داده‌ها را ۲۰ به ۸۰ می‌کنیم تا تفاوت آن را ببینیم شاید افزایش داده‌ها باعث افزایش دقت مدل شود.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=100, max_depth=15, random_state=42)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print("Train Accuracy:", train_accuracy)
print("Test Accuracy:", test_accuracy)
```

```
Train Accuracy: 0.8503777657852132
Test Accuracy: 0.8294964028776979
```

شکل (۳-۷) افزایش داده های یادگیری مدل

همین اتفاق هم افتاد و دقت تست افزایش یافت به سبب افزایش داده های یادگیری

```
classifier = DecisionTreeClassifier(min_samples_split=100, max_depth=14, random_state=42)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print("Train Accuracy:", train_accuracy)
print("Test Accuracy:", test_accuracy)
```

```
Train Accuracy: 0.8503777657852132
Test Accuracy: 0.8294964028776979
```

شکل (۳-۸) کاهش عمق درخت

تفاوت خاصی ایجاد نکرد.

ما در این گام بیش برآزش مدل را رفع کرده ایم و به دنبال افزایش دقت آن هستیم.

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=100, max_depth=14, random_state=42)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print("Train Accuracy:", train_accuracy)
print("Test Accuracy:", test_accuracy)

```

```

Train Accuracy: 0.8520147105852255
Test Accuracy: 0.8237410071942446

```

شکل (۹-۳) افزایش داده های یادگیری

دوباره افزایش داده های یادگیری دقت مدل ما را بیشتر کرد.

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=100, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)

```

```

Train Accuracy: 0.8463383434601854
Test Accuracy: 0.8284172661870504

```

شکل (۱۰-۳) افزایش عمق درخت به ۱۷

باز هم دقت تست را زیاد کرد و دقت ترین کمی کمتر شد و مشکلی نیست از بیش برآزش جلوگیری کرده ایم.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=100, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

```
Train Accuracy: 0.8453408886490376
Test Accuracy: 0.8352517985611511
```

شکل (۳-۱۱) افزایش نسبت داده ی تست به یادگیری ۰.۲

مشاهده می شود که با این کار هم بیش برآزش کمتر شد و دقت ما در تست بیشتر شد.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=100, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

```
Train Accuracy: 0.8478182659197204
Test Accuracy: 0.8333133469240916
```

شکل (۳-۱۲) افزایش نسبت داده ی تست به یادگیری ۰.۳

مشاهده می شود که در این حالت دقت تست از حالت پیشین کاهش پیدا کرد که دلخواه ما نیست پس نسبت قبل را ترجیح می دهیم.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=100, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

Train Accuracy: 0.8451096080970883
Test Accuracy: 0.8353719959706433

شکل (۳-۱۳) کاهش نسبت داده ی تست به ترین به ۰.۲۵

مشاهده می شود که این حالت نسبت به نسبت ۰.۲ دقت تست کمی بهتری داده است پس این نسبت را ترجیح می دهیم.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=110, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

Train Accuracy: 0.8441502374442366
Test Accuracy: 0.8376744855374874

شکل (۳-۱۴) افزایش حداقل سمپل جداسازی به ۱۱۰

باز هم دقت تست افزایش داشته که نشان می دهدشاخه هایی وجود داشته که زیادی تفکیک شده است و باید هرس می شده.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=120, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

Train Accuracy: 0.8434307094545977
Test Accuracy: 0.8382501079291984

شکل (۳-۱۵) افزایش حداقل جداسازی نمونه به ۱۲۰

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=130, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

Train Accuracy: 0.8427111814649589
Test Accuracy: 0.8386818247229817

شکل (۳-۱۶) افزایش حداقل جداسازی نمونه به ۱۳۰

همانطور که مشاهده می شود تا حد خیلی خوبی بیش برآزش مدل کنترل شده است اما دقت کم است و اگر آن را هم افزایش دهیم بسیار خوب می شود.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=150, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

Train Accuracy: 0.8420875905406053
Test Accuracy: 0.8405525974960426

شکل (۳-۱۷) افزایش جداسازی نمونه به ۱۵۰

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=200, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

Train Accuracy: 0.8396411953758335
Test Accuracy: 0.8353719959706433

شکل (۳-۱۸) افزایش جداسازی نمونه به ۲۰۰

پس از ۲۰۰ نمی توانیم تفکیک را بیشتر کنیم و عدد بهینه ی آن چیزی بین ۱۵۰ و ۲۰۰ است و پس از آزمایش مقدار های بیشتر ۱۵۰ به عنوان بهینه بدست آمد.

```
# city mapping
# filterdf['City'] = filterdf['City'].fillna(filterdf['City'].median())
# filterdf.loc[filterdf['City'] <= 2, 'City'] = 0
# filterdf.loc[(filterdf['City'] > 2) & (filterdf['City'] <= 6), 'City'] = 1
# filterdf.loc[(filterdf['City'] > 6) & (filterdf['City'] <= 11), 'City'] = 2
# filterdf.loc[(filterdf['City'] > 11) & (filterdf['City'] <= 16), 'City'] = 3
# filterdf.loc[(filterdf['City'] > 16) & (filterdf['City'] <= 21), 'City'] = 4
# filterdf.loc[(filterdf['City'] > 21) & (filterdf['City'] <= 29), 'City'] = 5
```

شکل (۳-۱۹) حذف میپینگ شهر


```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=150, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)

```

```

Train Accuracy: 0.8420875905406053
Test Accuracy: 0.8405525974960426

```

شکل (۳-۲۰) تاثیر حذف میپینگ شهرها

```

#####REMOVED#####
# Age mapping
# filterdf['Age'] = filterdf['Age'].fillna(filterdf['Age'].median())
# filterdf.loc[filterdf['Age'] <= 20 , 'Age'] = 0
# filterdf.loc[(filterdf['Age'] > 20) & (filterdf['Age'] <= 23), 'Age'] = 1
# filterdf.loc[(filterdf['Age'] > 23) & (filterdf['Age'] <= 25) , 'Age'] = 2
# filterdf.loc[(filterdf['Age'] > 25) & (filterdf['Age'] <= 28) , 'Age'] = 3
# filterdf.loc[(filterdf['Age'] > 28) & (filterdf['Age'] <= 31) , 'Age'] = 4
# filterdf.loc[(filterdf['Age'] > 31) & (filterdf['Age'] <= 34) , 'Age'] = 5
# filterdf

```

شکل (۳-۲۱) حذف میپینگ سن

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=150, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

Train Accuracy: 0.8405046289634001
Test Accuracy: 0.835515901568571

شکل (۳-۲۲) نتیجه حذف میپینگ سن

مشاهده می شود که میپینگ سن به درستی داشت به یادگیری مدل ما کمک می کرد و از بیش برآزش مدل جلوگیری می کرد پس این را به حالت قبل بر می گردانیم.

```
# Discretize CGPA into 20 groups
filterdf['CGPA_Group'] = pd.qcut(filterdf[nameToFilter], q=20, labels=range(20))
```

شکل (۳-۲۳) افزایش مقادیر و بازه های ستون CGPA

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=150, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

Train Accuracy: 0.8420875905406053
Test Accuracy: 0.8405525974960426

شکل (۳-۲۴) نتیجه ی افزایش بازه های ستون CGPA

```
# # Old Age mapping
# filterdf['Age'] = filterdf['Age'].fillna(filterdf['Age'].median())
# filterdf.loc[filterdf['Age'] <= 20 , 'Age'] = 0
# filterdf.loc[(filterdf['Age'] > 20) & (filterdf['Age'] <= 23), 'Age'] = 1
# filterdf.loc[(filterdf['Age'] > 23) & (filterdf['Age'] <= 25) , 'Age'] = 2
# filterdf.loc[(filterdf['Age'] > 25) & (filterdf['Age'] <= 28) , 'Age'] = 3
# filterdf.loc[(filterdf['Age'] > 28) & (filterdf['Age'] <= 31) , 'Age'] = 4
# filterdf.loc[(filterdf['Age'] > 31) & (filterdf['Age'] <= 34) , 'Age'] = 5
# filterdf

# Age mapping
filterdf['Age'] = filterdf['Age'].fillna(filterdf['Age'].median())
filterdf.loc[filterdf['Age'] <= 19 , 'Age'] = 0
filterdf.loc[(filterdf['Age'] > 19) & (filterdf['Age'] <= 21), 'Age'] = 1
filterdf.loc[(filterdf['Age'] > 21) & (filterdf['Age'] <= 23) , 'Age'] = 2
filterdf.loc[(filterdf['Age'] > 23) & (filterdf['Age'] <= 25) , 'Age'] = 3
filterdf.loc[(filterdf['Age'] > 25) & (filterdf['Age'] <= 27) , 'Age'] = 4
filterdf.loc[(filterdf['Age'] > 27) & (filterdf['Age'] <= 29) , 'Age'] = 5
filterdf.loc[(filterdf['Age'] > 29) & (filterdf['Age'] <= 31) , 'Age'] = 6
filterdf.loc[(filterdf['Age'] > 31) & (filterdf['Age'] <= 33) , 'Age'] = 7
filterdf.loc[(filterdf['Age'] > 33) & (filterdf['Age'] <= 34) , 'Age'] = 7
filterdf
```

شکل (۳-۲۵) باز تعریف گروه های سنی دوباره به بازه های کوچک تر

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=150, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)
```

```
Train Accuracy: 0.841655873746822
Test Accuracy: 0.8352280903727155
```

شکل (۳-۲۶) نتیجه و دقت پس از بازتعریف دوباره ی گروه های سنی

```
# Age mapping
filterdf['Age'] = filterdf['Age'].fillna(filterdf['Age'].median())
filterdf.loc[filterdf['Age'] <= 23 , 'Age'] = 0
filterdf.loc[(filterdf['Age'] > 23) & (filterdf['Age'] <= 28) , 'Age'] = 1
filterdf.loc[(filterdf['Age'] > 28) & (filterdf['Age'] <= 31) , 'Age'] = 2
filterdf.loc[(filterdf['Age'] > 31) & (filterdf['Age'] <= 34) , 'Age'] = 3
```

شکل (۳-۲۷) افزایش بازه های سنی

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

classifier = DecisionTreeClassifier(min_samples_split=150, max_depth=17, num_splits=1000)
classifier.fit(X_train, y_train)

train_predictions = classifier.predict(X_train)
test_predictions = classifier.predict(X_test)

train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

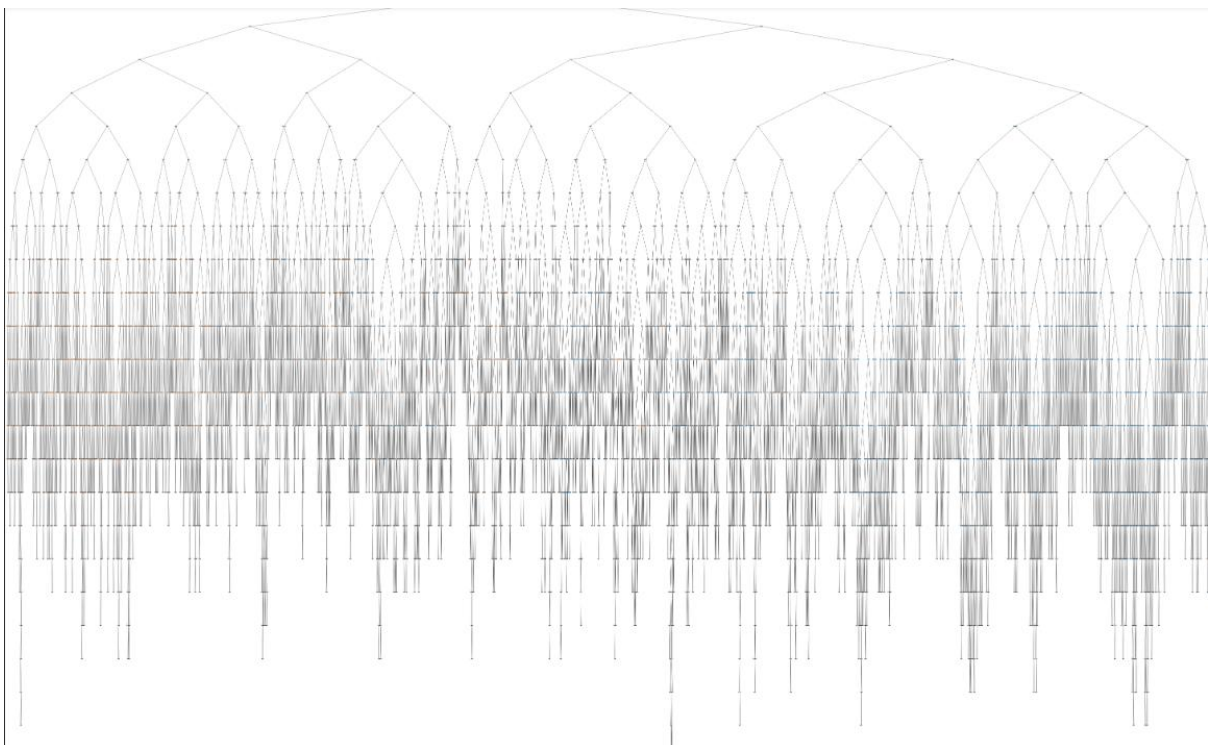
print('Train Accuracy:', train_accuracy)
print('Test Accuracy:', test_accuracy)

```

Train Accuracy: 0.8412241569530388
Test Accuracy: 0.8376744855374874

شکل (۳-۲۸) نتیجه ی افزایش بازه های سنی

مشاهده می شود که با تغییر بازه های سنی به دقت بالاتری نمی رسیم.



شکل (۳-۲۹) چاپ درخت تصمیم

فصل ٤:

مراجع

مراجع

[۱] اسلاید درس

Chat GPT 4o [۲]

geeksforgeeks.org [۳]